# Redes de Sensores Biomédicos

## Implementação de um Sistema de Aquisição de Dados

Inês Pires(1), Lícia Almeida(2), Pedro Prata(3), Joaquim Alves(4), Nuno Gueiral(5)
(1)1200732@isep.ipp.pt, (2)1211459@isep.ipp.pt, (3)1211122@isep.ipp.pt,
(4)jaa@isep.ipp.pt, (5)ndg@isep.ipp.pt
Instituto Superior de Engenharia do Porto, Porto, Portugal

**Resumo —** A aquisição de dados é essencial para a análise de fenómenos físicos, permitindo a conversão de sinais elétricos em dados digitais através de conversores analógico-digitais (ADC). O presente relatório, desenvolvido no contexto da Unidade Curricular de Redes de Sensores Biomédicos, apresenta um protótipo de aquisição de dados para monitorizar a temperatura corporal (25-50 ºC) usando um sensor NTC, ponte de *Wheatstone*, amplificadores TL084 e conversores ADC. O sistema converte variações de resistência em tensão, amplifica e ajusta o sinal para o intervalo de 0 a 5 V. A calibração com um termómetro de referência resultou numa relação tensão-temperatura precisa ($R^2$=0,99), apesar de apresentar limitações de linearidade acima de 39 ºC. Também foram integradas comunicações sem fios: ZigBee (módulo XBee), Wi-Fi (ESP32) e Bluetooth (HC-05), viabilizando aplicações em *Internet of Things*.

**Palavras-Chave —** DAQ, sensor NTC, temperatura, tensão, calibração, Arduino, ZigBee, ESP32, Wi-Fi, Bluetooth

# 1 Introdução

Nesta secção é feita uma introdução teórica à primeira parte do projeto implementado, referente à projeção de um sistema de aquisição de dados.

### 1.1 Sistema de Aquisição de Dados

A aquisição de dados (DAQ) adquire um papel de extrema importância na interpretação e análise de fenómenos físicos do mundo real, permitindo o registo preciso de parâmetros como temperatura, som e pressão. Este processo envolve a conversão de sinais elétricos, medidos por sensores e transdutores, em formato digital através de conversores analógico-digitais (ADC). A DAQ é fundamental na obtenção de dados confiáveis e no acompanhamento de ensaios, em tempo real, aumentando a precisão e eficiência nas operações laboratoriais, de acordo com [1].

Os transdutores e sensores são componentes essenciais no processo de DAQ, já que transformam eventos físicos em sinais elétricos compatíveis com o *hardware* de condicionamento de sinal. Este condicionamento envolve pré-processamento, acerto de nível, amplificação, filtragem, excitação e linearização dos sinais elétricos para garantir que sejam adequados para o sistema de aquisição de dados. A eficácia do equipamento de condicionamento de sinal depende de diversos fatores, como o tipo e número de transdutores, necessidades de excitação e ligação à massa, como mencionado na Ref. [2].

### 1.2 Sensor NTC

Os termístores são uma classe de sensores de temperatura baseados no efeito termorresistente. Inclui óxidos metálicos, exibindo diferentes valores de coeficientes térmicos de resistência na faixa de temperatura ambiente e moderada. Geralmente o seu coeficiente de temperatura de resistência pode ser positivo ou negativo, PTC e NTC, respetivamente, como referido em [3]. Um sensor NTC (*Negative Temperature Coefficient*) é um tipo de termístor que apresenta uma resistência à medida que a temperatura aumenta, como demonstra a Fig. 1. Por outro lado, o PTC é um termístor que funciona como fusível ou para proteção de corrente e não como sensor. Uma característica importante de medição de temperatura dos termístores é a sua sensibilidade extremamente alta, o que os torna bastante úteis para medições de temperatura em

equipamentos eletrónicos, ar condicionado, carros, eletrodomésticos, etc., uma vez que a mudança na resistência pode ser facilmente convertida numa leitura de temperatura, explicado em [3][4].

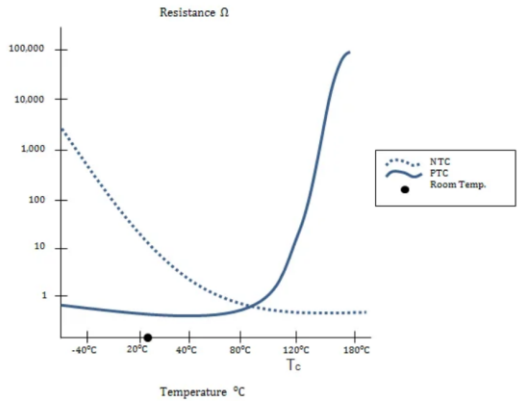



**Fig. 1.** Curva característica de temperatura de um termístor [5].

Esta classe de sensores, comparativamente com termopares, apresenta a vantagem de apresentarem pequenas dimensões, baixo preço, pois são produzidos em massa como componentes eletrónicos com um par de fios ou fios curtos.

A resistência pode ser traduzida pela Eq. 1, na qual $R_0$ é a resistência nominal do termístor medida em $\theta_0$ = 230 K = 20 ºC, e, β é o coeficiente de θ com valores entre 3000 e 5000, como referido em [3].

$$R(\theta) = R_0 e^{\beta\left(\frac{1}{\theta}-\frac{1}{\theta_0}\right)} \tag{1}$$

### 1.3 Visão Geral e Objetivo

O presente relatório foi desenvolvido no âmbito da Unidade Curricular de Redes de Sensores Biomédicos, lecionada no primeiro ano do Mestrado em Engenharia Biomédica, e descreve o desenvolvimento de um circuito de um sistema de aquisição de dados, capaz de monitorizar a temperatura ambiente, num intervalo de 25 a 50 graus Celsius. A monitorização precisa da temperatura assume uma grande importância na área da saúde, onde a supervisão constante e exata deste parâmetro garante a segurança e eficácia dos dispositivos médicos e instalações de assistência à saúde. A realização deste projeto teve, então, como principal objetivo o desenvolvimento de um protótipo de um sistema de aquisição para a medição da temperatura, em tempo real, utilizando um circuito constituído por um sensor NTC, resistências, três potenciómetros e dois amplificadores operacionais. O diagrama de blocos do circuito desenvolvido, para o sistema de DAQ, encontra-se na Fig. 2.

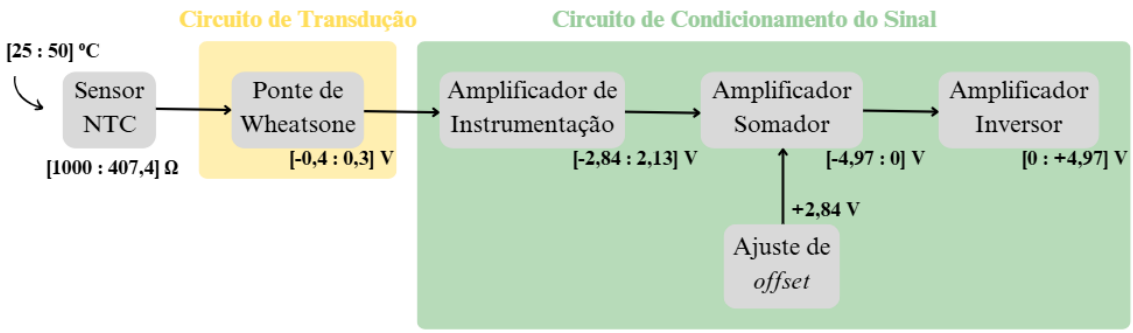



**Fig. 2.** Diagrama de blocos do circuito implementado.

# 2 Implementação do Sistema

Nesta secção é descrita, por partes, a implementação do DAQ completo.

## 2.1 Ponte de *Wheatstone*

A ponte de *Wheatstone* é um circuito usado para converter variações de resistência em tensão. No sistema de medição de temperatura, um sensor resistivo (NTC) é colocado na ponte, e, com a mudança de temperatura, a resistência do sensor varia, desequilibrando a ponte e gerando uma diferença de tensão. Essa tensão é proporcional à temperatura e pode ser facilmente medida e processada no Sistema de aquisição de dados.

O intervalo de temperatura pretendido é [25:50] ℃ que pela análise do *datasheet* corresponde a [1000:407,4] Ω. Para traduzir este intervalo foi então necessário aplicar a ponte de *Wheatstone* (Fig. 3), onde, $R_1 = R_2 = 2200\ \Omega$ e $R_{pot} = 651{,}1\ \Omega$, valor obtido pela Eq. 2, de maneira a equilibrar a ponte para que o valor de referência seja 36,5 ℃.

$$R(T) = R_{ref} \times e^{\left(A+\frac{B}{T}+\frac{C}{T^2}+\frac{D}{T^3}\right)}, \tag{2}$$

$$A = -21.0704, B = 11903.95, C = -2504699, D = 247033800$$

Com $R_1 = R_2 = 2200\ \Omega$, $R_{pot} = 651{,}1\ \Omega$, e o intervalo de $R_{sensor}$ variando de [1000:407,4] Ω, podemos calcular as tensões mínima e máxima de saída da ponte de *Wheatstone* pela Eq. 3.

$$V_{AB} = V_0 \times \frac{R_1 \times \Delta R_{sensor}}{(R_1+R_{pot})(R_2+R_{sensor}+\Delta R_{sensor})} \tag{3}$$

$$V_{AB\ a\ 50℃} = 5 \times \frac{2200 \times (651{,}1 - 407{,}4)}{(2200 + 651{,}1)(2200 + 651{,}1 + (651{,}1 - 407{,}4))} = 0{,}30\ V$$

$$V_{AB\ a\ 25℃} = 5 \times \frac{2200 \times (651{,}1 - 1000)}{(2200 + 651{,}1)(2200 + 651{,}1 + (651{,}1 - 1000))} = -0{,}42\ V$$

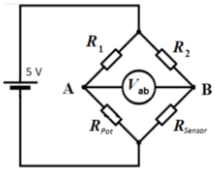



**Fig. 3.** Circuito de transdução - Ponte de *Wheatstone.*

## 2.2 Amplificador de Instrumentação

Tendo em conta que, num intervalo compreendido entre os 25 ºC e os 50 ºC, a diferença de potencial entre os pontos A e B corresponde a 0,30 V e - 0,42 V, respetivamente, torna-se imperativo amplificar o sinal para que, no final do DAQ, este esteja compreendido num intervalo [0; 5] V. Para isso, recorreu-se ao Amplificador Operacional TL084, cujas condições recomendadas de utilização encontram-se na Fig. 4. O TL084 possui quatro amplificadores operacionais independentes dentro de si, que, apesar de compartilharem a mesma alimentação, possuem entradas e saídas separadas, permitindo a sua utilização em diferentes partes do circuito.

| | | | MIN | MAX | UNIT |
|---|---|---|---|---|---|
| $V_{CC+}$ | Supply voltage | | 5 | 15 | V |
| $V_{CC-}$ | Supply voltage | | –5 | –15 | V |
| $V_{CM}$ | Common-mode voltage | | $V_{CC-}$ + 4 | $V_{CC+}$ – 4 | V |
| $T_A$ | Ambient temperature | TL08xM | –55 | 125 | °C |
| | | TL08xQ | –40 | 125 | |
| | | TL08xI | –40 | 85 | |
| | | TL08xC | 0 | 70 | |

**Fig. 4.** Condições recomendadas de funcionamento do AMPOP TL084.

Deste modo, o mesmo foi alimentado a + 5 V e a - 5 V nos terminais VCC+ e VCC-, respetivamente, condições que, apesar de serem mínimas, encontram-se dentro do intervalo estipulado pelo fabricante, tal como indica a Fig. 4. Estas tensões foram criadas a partir da utilização de duas fontes de alimentação previamente configuradas para alimentar 5 V ao AMPOP. Assim, garantiu-se que ambas estavam conectadas em série, ligando-se o terminal positivo de uma fonte A ao terminal negativo de uma B, criando-se assim um ponto de referência (massa). Deste modo, o terminal negativo da fonte A fornecerá os - 5 V em relação ao ponto comum, enquanto que o ponto de conexão será de 0 V e o terminal positivo da fonte B garantirá os + 5 V, também em relação ao mesmo ponto.

Assim, foi implementado o circuito da Fig. 5, onde $V_A$ e $V_B$ correspondem, respetivamente, aos pontos A e B da ponte de *Wheatstone*.

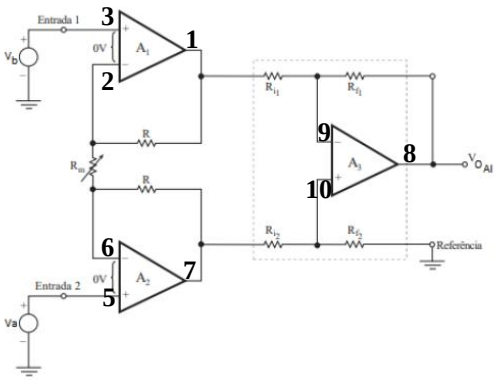



**Fig. 5.** Amplificador de Instrumentação.

Sendo $V_{AB}$, num intervalo de 25 ºC a 50 ºC, igual a aproximadamente [-0,4; 0,3] V, o ganho que este amplificador de instrumentação terá que ter, de modo a que, à saída do Sistema de Aquisição de Dados, o sinal esteja compreendido entre [0; 5] V, é dado pela Eq. 4:

$$G = \frac{5}{0{,}7} = 7{,}1 \tag{4}$$

Encontrado o ganho, torna-se possível determinar o valor de $R_m$ de modo a que o ganho seja o indicado, através do uso da Eq. 5.

$$V_{O_{AI}} = \frac{R_f}{R_i}\left(1 + \frac{2R}{R_m}\right)(V_A - V_B) \tag{5}$$

Assim, a partir da equação anterior (Eq. 5), retira-se que:

$$G = 7{,}1 = \frac{R_f}{R_i}\left(1 + \frac{2R}{R_m}\right)$$

Assumindo, de modo a facilitar os cálculos, que $R_f = R_i = 1$ kΩ, e atribuindo um valor arbitrário a R de, por exemplo, 2,2 kΩ, tem se que:

$$7{,}1 = 1 + \frac{2 \times 2200}{R_m} \leftrightarrow R_m = 721{,}3\ \Omega$$

Deste modo, e novamente a partir da Eq. 5, torna-se possível calcular os valores expectáveis à saída do amplificador de instrumentação para as temperaturas de 25 ºC e 50 ºC.

$$V_{OAI(25\,ºC)} = 7{,}1 \times (-0{,}4) = \mathbf{-2{,}84\ V}$$

$$V_{OAI(50\,ºC)} = 7{,}1 \times 0{,}3 = \mathbf{2{,}13\ V}$$

### *2.3* Definição da tensão de *offset*

Sabendo que à saída do amplificador de instrumentação o intervalo foi [-2,84; 2,13] V, como referido anteriormente, foi necessário aplicar um valor de *offset* de +2.84 V, para ajustar esse intervalo de forma a obter-se [0; 5] V.

Assim, foi implementado o circuito da Fig. 6. De acordo com a Eq. 6, obteve-se o valor para o potenciómetro ($R_4$) de aproximadamente 894 Ω, considerando que a resistência $R_3$ utilizada tinha um valor de 2200 Ω. Este ajuste permitiu que o sinal final se alinhasse ao intervalo exigido pelo sistema de aquisição de dados, evitando saturação e aumentado a precisão do mesmo.

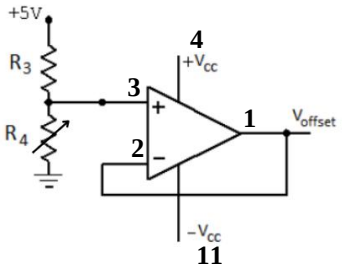



**Fig. 6.** Divisor de tensão para obter a tensão *offset*.

$$V_{offset} = 5\frac{R_4}{R_3+R_4} \Leftrightarrow 2{,}84 = 5\frac{R_4}{2200+R_4} \Leftrightarrow R_4 = 894{,}07\ \Omega \qquad (6)$$

### 2.4 Amplificador Somador

Um amplificador somador é um componente que permite somar a tensão offset à saída do amplificador instrumental. Essa adição é essencial para ajustar o nível do sinal de saída, garantindo que ele esteja dentro da faixa operacional desejada, obtendo-se o intervalo [-4,97: 0] V, Eq. 7. Com esta montagem (Fig. 7), o sistema de medição pode fornecer leituras mais precisas e confiáveis, fundamentais para aplicações na monotorização da saúde do indivíduo e controlo de temperatura.

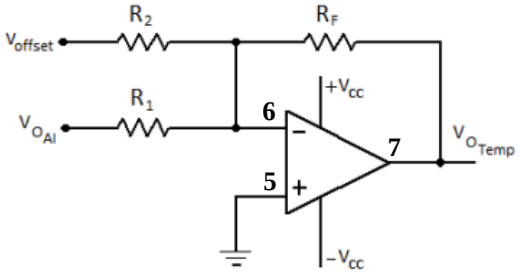



**Fig. 7.** Amplificador somador para somar a tensão offset à saída do amplificador instrumental

$$V_{0\,Temp} = -R_F\left(\frac{V_{0AI}}{R_1} + \frac{V_{offset}}{R_2}\right) \tag{7}$$

$$V_{0\,Temp\,(a\,25\,°C)} = -3300\left(\frac{-2{,}84}{3300} + \frac{2{,}84}{3300}\right) = 0\,V$$

### 2.5 Amplificador Inversor

Para alcançar o intervalo de [0: 4,97] V desejado, foi aplicado um somador inversor ao circuito (Fig. 8). Esta configuração permite que os valores de tensão negativos sejam transformados em valores positivos, visto que como $R_F = R_1$ o seu ganho é de -1. Com esta abordagem, o sistema de aquisição de dados está então apto para a medição da temperatura corporal, que em média é de 36,5 °C.

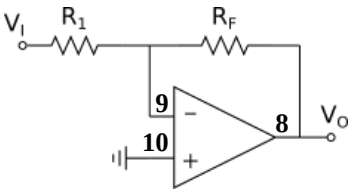



**Fig. 8.** Amplificador inversor para obter o intervalo pretendido.

### 2.6 Circuito Final

Obtemos então por fim o nosso circuito final, evidenciado na Fig. 9, pronto a ser ligado a um ADC.

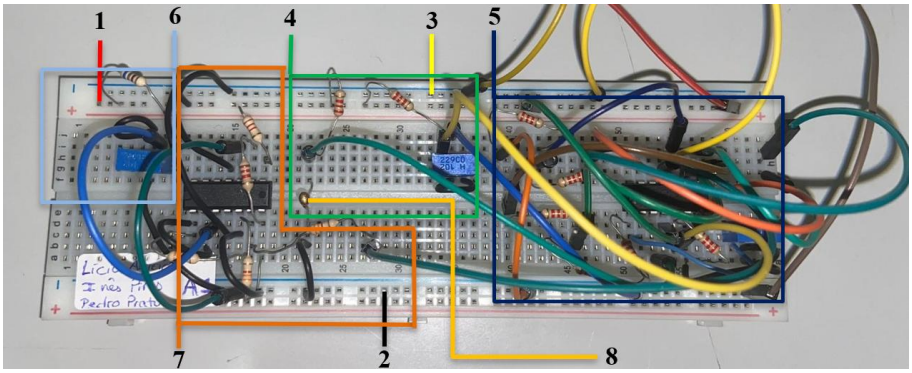



**Fig. 9.** Circuito final legendado.

**Legenda da Fig. 9:**

1 – + 5 V
2 – GND (massa)
3 – - 5 V
4 – Ponte de Wheatstone
5 – Amplificador de Instrumentação
6 – Tensão *Offset*
7 – Amplificador Somador e Inversor
8 – Sensor NTC

## 3 Calibração

A calibração de instrumentos de medição é um método imprescindível na garantia da precisão e conformidade com especificações técnicas e sistemas de qualidade dos mesmos. De acordo com o Vocabulário Internacional de Metrologia, a definição de calibração é a "operação que estabelece, sob condições especificadas, num primeiro passo, uma relação entre os valores e incertezas de medição fornecidos por padrões e as indicações correspondentes com as incertezas associadas; num segundo passo, utiliza esta informação para estabelecer uma relação visando a obtenção dum resultado de medição a partir duma indicação",

referido em [5]. Contudo, a calibração por si só não garante a precisão das medições, isto é, apenas identifica os erros do equipamento. A análise desses erros e as decisões subsequentes fazem parte da confirmação metrológica, conforme definido pela norma ISO 10012, que visa garantir a conformidade do equipamento com os requisitos de uso. Para além disso, alguns equipamentos estão sujeitos a regras específicas, conhecidas como controlo metrológico ou metrologia legal, devido ao impacto que têm sobre os cidadãos, tendo que ser submetidos a uma verificação, como mencionado em [6].

As calibrações definidas pelo Instituto Português de Qualidade (IPQ) utilizam pontos fixos bem definidos da Escala Internacional de Temperatura de 1990 (EIT90), de acordo com a Ref. [7], padrões de resistência e instrumentos de alta precisão para garantir a rastreabilidade e a precisão das medições da temperatura, sendo um processo rigorosamente controlado e documentado para assegurar a conformidade com os padrões internacionais e fornecer suporte técnico necessário para diversas aplicações na metrologia legal e industrial. Comparando a abordagem da aula com as práticas estabelecidas pelo IPQ é evidente que ambos os métodos visam garantir a precisão e rastreabilidade das medições de temperatura. O IPQ, ao seguir normas internacionais reconhecidas, enfatiza a importância de uma calibração rigorosa dos instrumentos, enquanto que a calibração aplicada aos dados recolhidos adota uma metodologia mais prática e específica para um ambiente controlado de laboratório.

Para a calibração foi utilizado um recipiente com água, em cima duma placa de aquecimento, onde foram colocados o sensor NTC e o termómetro até o sistema atingir o equilíbrio térmico, e aí, aqueceu-se o recipiente gradualmente, de forma a registar as temperaturas em intervalos de 0,5 ºC, registando se as tensões à saída do amplificador inversor, lidas no multímetro. O esquema de montagem encontra-se na Fig. 10.

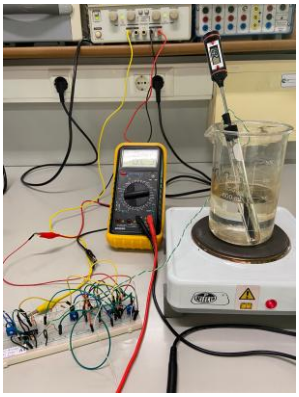


**Fig. 10.** Esquema de montagem utilizado na calibração do circuito.

O gráfico de calibração obtido (Fig. 11) mostra que à medida que a temperatura aumenta, a tensão aumenta, o que era esperado, uma vez que, com o aumento da temperatura, a resistência do sensor diminui, consequentemente, a tensão entre o ponto B e a referência ($V_B$) diminui, e, sendo a tensão $V_{AB} = V_A - V_B$, então $V_{AB}$ aumenta, bem como, a tensão à saída do ampop inversor.

No entanto, embora o aumento inicial seja aproximadamente linear, a partir de cerca de 39 ºC a curva mostra uma tendência de estagnação, tornando-se constante. Comportamento este pode estar relacionado com uma configuração incorreta do *offset* ou à limitação dos amplificadores operacionais que foram alimentados a +5 V, tendo o sistema ficado saturado antes de se atingir esse valor de tensão à saída. Para se obter um valor de tensão de +5 V a 50 ºC, o circuito deveria ter sido alimentado por um valor maior do que esse. Os valores inesperados de tensão negativa podem dever-se a uma incorreta implementação do ajuste de nível, resultando em valores fora do intervalo pretendido.

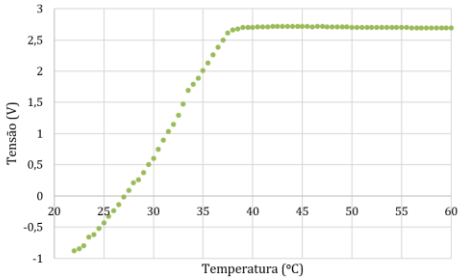



**Fig. 11.** Gráfico Tensão vs Temperatura obtido originalmente.

Desta forma, foram traçados os gráficos Tensão *vs* Temperatura (Fig. 12 a)) e Temperatura *vs* Tensão (Fig. 12 b)), para os pontos anteriores à estagnação da tensão, para se avaliar a linearidade do circuito. É importante referir que os valores obtidos para os quadrados do coeficiente de correlação são ambos 0,9925, valor muito próximo de 1, pelo que indica a elevada qualidade de ajuste de linha aos dados. A equação obtida a partir das curvas de calibração traduz a relação entre a tensão, em V, à saída do ampop inversor e a temperatura, em ºC.

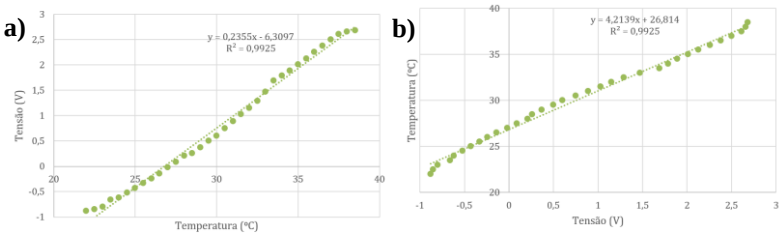



**Fig. 12.** Gráficos a) Tensão vs Temperatura, e, b) Temperatura vs Tensão, com linha de tendência.

# 4 Arduino

O Arduino, uma plataforma eletrónica de código aberto, tornou-se a escolha ideal para quem desenvolve projetos interativos de *hardware* e *software*. Uma placa Arduino (como a Uno), conectada a uma *protoboard* com componentes como entradas, sensores, luzes e *displays*, pode ser controlada por um código escrito no ambiente de desenvolvimento Arduino IDE. Para isso, é necessário criar protótipos com o Arduino, uma prática que tem adquirido popularidade à medida que a plataforma se expande. Contudo, a criação de protótipos com Arduino pode ser desafiadora para aqueles que não são programadores, mas que se interessam por essa área. Com o crescente interesse do público, surge a necessidade de disponibilizar informações de fácil acesso e compreensão [8].

## 4.1 Características e funcionamento

Nesta secção serão apresentadas as características e o princípio de funcionamento do Arduino.

### 4.1.1. Hardware

Existem várias placas Arduino, mas nesta análise, focaremos na Arduino Uno. O Arduino Uno é uma placa de microcontrolador baseada no ATmega328, um microcontrolador de alto desempenho, baseado na arquitetura AVR RISC de 8 *bits* da *Atmel*. A placa opera com tensões de 1,8 a 5,5 volts, oferecendo 14 pinos de entrada/saída digitais, 6 entradas analógicas, e um oscilador de 16 MHz. Além disso, inclui uma conexão USB, um conector de alimentação, um conector ICSP e um botão de *reset*, proporcionando todos os recursos necessários para o desenvolvimento de projetos interativos [9].

A placa pode ser alimentada por uma conexão USB ou por uma fonte externa, como uma bateria ou um adaptador AC-DC. Quando alimentada externamente, a tensão de operação pode variar entre 6-20 V através do pino de alimentação (Vin) [9].

O Arduino Uno oferece 14 pinos digitais, que podem ser configurados como entradas ou saídas, com 6 pinos adicionais que possuem funções PWM (modulação por largura de pulso). Estes pinos operam com uma tensão de 5 V e são programados usando funções como *pinMode()*, *digitalWrite()* e *digitalRead()*. Além disso, a placa tem 6 pinos analógicos que permitem a leitura de sinais analógicos de sensores, como temperatura e luminosidade [9].

O Arduino Uno facilita a comunicação com outros dispositivos através de sua capacidade de comunicação serial. A biblioteca *SoftwareSerial* permite a comunicação em qualquer um dos pinos digitais, o que expande a flexibilidade da placa na troca de dados com outros sistemas.

O microcontrolador ATmega328 da Arduino Uno possui 32 KB de memória flash, 2 KB de RAM e 1 KB de EEPROM. Ele opera com 5V, e tem 14 pinos de I/O que são capazes de realizar operações digitais e analógicas. A arquitetura interna do microcontrolador inclui: CPU (Unidade Central de Processamento) que executa as instruções do código; memória, a RAM para dados temporários e a ROM para armazenamento permanente do código; e, conversor A/D para conversão de sinais analógicos em digitais, com uma resolução de 10 *bits* [9].

### 4.1.2. Software

O Arduino IDE é um ambiente de desenvolvimento de código aberto, projetado para ser intuitivo e fácil de usar. Este é escrito em Java e baseado no *Processing*, sendo compatível com sistemas operacionais como Windows, Mac OS X e Linux. A IDE oferece funcionalidades como realce de sintaxe, casamento de chaves e indentação automática, facilitando a programação, especialmente para iniciantes [10].

A linguagem utilizada no Arduino é C/C++. O código desenvolvido, conhecido como *sketch*, pode ser facilmente escrito e carregado para a placa. A linguagem permite controlar pinos de entrada/saída, realizar cálculos matemáticos e interagir com dispositivos conectados [10].

O Arduino IDE inclui várias bibliotecas que facilitam a implementação de funcionalidades avançadas, como controle de motores, leitura de sensores e comunicação entre dispositivos [10].

O processo de desenvolvimento na Arduino começa com a escrita do código, ou *sketch*, que é compilado e carregado para a placa através da conexão USB. A IDE também oferece um monitor serial, permitindo que os dados, como leituras de sensores ou mensagens, sejam visualizados em tempo real. A execução do código começa com a função *setup()*, que configura os parâmetros iniciais, como os pinos de entrada/saída. Após isso, a função *loop()* é executada continuamente, realizando as operações programadas, como a leitura de sensores e o controle de atuadores, até que o programa seja interrompido ou finalizado [10]. A IDE do Arduino permite a depuração do código através do monitor serial, onde é possível visualizar valores de variáveis, dados dos sensores e mensagens de erro durante a execução. Esse recurso é útil para testar e ajustar o programa, garantindo que a interação com os dispositivos conectados seja feita corretamente. Além de ser utilizada diretamente no Arduino IDE, a programação pode ser integrada com outras plataformas externas, como Plataformas de *Internet of Things* (IoT). Por exemplo, *Blynk* e *ThingSpeak* permitem que os dispositivos conectados ao Arduino sejam controlados e monitorados remotamente através da internet [10].

Em suma, o *software* do Arduino, juntamente com o *hardware* intuitivo, oferece uma plataforma poderosa para desenvolvedores tanto iniciantes como experientes. A combinação da facilidade de uso, flexibilidade e suporte comunitário permite que qualquer pessoa, crie e teste rapidamente novos projetos.

### 4.1.3. Bibliotecas e aquisição

Nesta secção, são apresentadas as bibliotecas utilizadas para a implementação do sistema. As bibliotecas simplificam a comunicação com os sensores e módulos de transmissão de dados, permitindo a recolha e envio eficiente das medições. Além disso, são detalhadas as funções específicas empregadas para processar e exibir as informações adquiridas.

Uma das bibliotecas pilares neste projeto é a DHT.h, que é usada para interagir com os sensores de temperatura e humidade DHT. Além disso, a Xbee.h marca presença também neste projeto, dado que permite a comunicação e o controlo das mensagens recebidas em comunicações sem fio usando módulos XBee. A binary.h, por sua vez, inclui definições para converter números em binário, manipular bits, ou ainda formatar saídas binárias, enquanto que a Printers.h facilita a exibição de mensagens formatadas e exibir os dados no serial monitor. Existem ainda as bibliotecas Wifi.h, a ESPAsyncWebServer.h e a Adafruit_Sensor.h. A primeira é usada para habilitar a conectividade Wi-Fi em placas ESP32, enquanto que a segunda permite, tal como o nome indica, a criação de um servidor assíncrono para o ESP32, facilitando a configuração de páginas e APIs para o controlo e monitorização remota do dispositivo. A terceira, por fim, fornece uma *interface default* para trabalhar com os sensores da Adafruit. Finalmente, as bibliotecas nRF24L01 e RF24.h facilitam a comunicação sem fio entre dispositivos usando módulos nRF24L01.

Na Figura 13 está representado o código responsável por medir a temperatura e a humidade utilizando um sensor DHT e transmitindo esses mesmos dados através de um módulo XBee.

```
1  void setup() {
2  Serial.begin(9600);
3  dht.begin();
4  xbee.begin(Serial);
5  delay(1);
6  xbee.onPacketError(printErrorCb, (uintptr_t)(Print*)&Serial);
7  xbee.onResponse(printErrorCb, (uintptr_t)(Print*)&Serial);}
8  void sendPacket() {
9  ZBTxRequest txRequest;
10 txRequest.setAddress64(0x0013A20040A146B8); //
11 AllocBuffer<9> packet;
12 packet.append<uint8_t>(1);
13 packet.append<float>(dht.readTemperature());
14 packet.append<float>(dht.readHumidity());
15 txRequest.setPayload(packet.head, packet.len());
16 xbee.send(txRequest);}
17 void loop() {
18 float h = dht.readHumidity();
19 float t = dht.readTemperature();
20 delay(3600);
21 Serial.println(" ");
22 Serial.print("Temperature: ");
23 Serial.print(t);
24 Serial.println(" C");
25 Serial.print("Humidity: ");
26 Serial.print(h);
27 Serial.println(" %");
28 sendPacket();
29 xbee.loop();}
```

**Fig. 13.** Código responsável por medir a temperatura e a humidade com um sensor DHT11.

A função *setup()* assegura a comunicação inicial, onde o sensor DHT é inicializado e o módulo XBee é configurado, definindo-se também callbacks para lidar com erros e respostas recebidas do XBee.

A função *sendPacket()* é de importância extrema, dado que esta intervém diretamente na transferência de dados para o módulo XBee. É nesta função que é criado um objeto ZBTxRequest, de modo a configurar o endereço de destino do pacote, que é definido como um endereço hexadecimal específico. Além disso, é alocado um buffer de 9 bytes que armazena a temperatura e a humidade que vão ser enviadas via XBee.

Por último, a função *loop()*, que é executada continuamente, armazena os valores da temperatura e da humidade em variáveis que são exibidas no monitor serial. Após um *delay* de 3600 ms (3,6 s), a função *sendPacket()* é novamente chamada para transmitir os dados atualizados. Por fim, a função xbee.loop() é executada, permitindo que o módulo XBee processe os pacotes recebidos ou então respostas pendentes.

# 5 Sensor DHT11

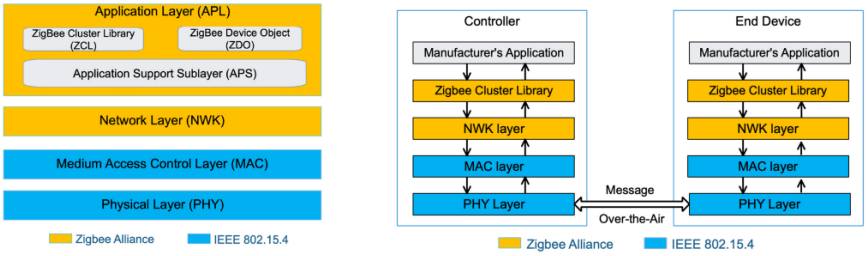
O DHT11 é um sensor digital que mede tanto a temperatura como a humidade. É compacto, económico e fácil de interagir com microcontroladores devido à sua saída digital, o que o torna popular. Este utiliza um sensor de humidade capacitivo e um termístor para fornecer medições precisas, com uma gama de temperaturas de 0 a 50 °C e uma incerteza de ± 2 %. É conhecido pelo seu tempo de resposta rápido, boa resistência a interferências e baixo consumo de energia, funcionando numa gama de tensão de 3,5-5 V [11].

A principal diferença entre o DHT11 e os sensores analógicos como o NTC reside na sua saída e funcionalidade. O DHT11 fornece uma saída digital, o que simplifica a interface com microcontroladores e sistemas de aquisição de dados, enquanto sensores analógicos como o NTC produzem uma tensão que é linearmente proporcional à temperatura, exigindo conversão adicional para sistemas digitais. Além disso, o DHT11 pode medir tanto a temperatura como a humidade, oferecendo uma dupla funcionalidade num único dispositivo, ao contrário do NTC, que apenas mede a temperatura [12].

O sensor DHT11 é amplamente utilizado em aplicações em que uma precisão moderada é suficiente, como a monitorização da temperatura ambiente, incubadoras e monitorização ambiental em estufas. O seu pequeno tamanho e baixo custo tornam-no uma opção atrativa para projetos que não exijam uma precisão elevada. No entanto, a sua gama de temperaturas limitada e a sua menor exatidão em comparação com alguns outros sensores podem ser um inconveniente em aplicações que exijam medições precisas [11].

# 6 Comunicação wireless ZigBee

O ZigBee é um protocolo de comunicação pessoal sem fios (WPAN) eficiente em termos de recursos, e amplamente utilizado em dispositivos IoT. Baseia-se no padrão IEEE 802.15.4, e destaca-se pelo baixo consumo de energia. A sua estrutura é constituída por camadas como Física e MAC responsáveis pela transmissão de pacotes, e camadas de Rede e Aplicação, definidas pela ZigBee Alliance, que gerem funcionalidades de alto nível e pacotes na rede (Fig. 14), de acordo com [13].




(a) Zigbee protocol stack architecture [5]

(b) A prototype of message exchange process between two Zigbee end devices [28]

**Fig. 14.** Estrutura geral da pilha de protocolos Zigbee e respetivo processo de comunicação [13].

## 6.1 Componentes físicos do ZigBee

Os Dispositivos Totalmente Funcionais (FFDs) são capazes de desempenhar todas as funções especificadas pela norma ZigBee, incluindo tarefas de encaminhamento, coordenação e monitorização. Dependendo do seu objetivo na rede, um FFD pode atuar como coordenador, encaminhador ou dispositivo terminal (que pode ser um FFD ou um RFD). Estes dispositivos estão sempre ligados e conectados à rede sem fios, normalmente derivando a sua energia da própria rede em sistemas ZigBee. Os FFDs asseguram o

funcionamento correto e a comunicação dentro da rede ZigBee, servindo como componentes cruciais na manutenção das operações da rede [14].

Os Dispositivos de Função Reduzida (RFDs) são concebidos com capacidades computacionais limitadas em comparação com os FFDs. A principal função de um RFD é ligar-se a uma rede existente, verificar se os dados estão disponíveis para transmissão e, em seguida, solicitar que os dados sejam enviados pelo coordenador da rede. Para conservar a vida da bateria, os RFDs entram normalmente em modo de suspensão quando não estão a ser utilizados. Uma vez que os RFDs não podem encaminhar dados, têm de depender dos FFDs para facilitar a comunicação. Exemplos comuns de RFDs incluem sensores e atuadores que recolhem dados (por exemplo, temperatura, níveis de luz) ou controlam outros dispositivos dentro do sistema [14].

## 6.2 Componentes lógicos do ZigBee

O **coordenador** desempenha um papel central nas redes ZigBee, uma vez que é responsável por estabelecer e definir os parâmetros da rede, incluindo a topologia, o tamanho dos pacotes e outras configurações críticas. Também atua como *gateway* através do qual os dispositivos externos se podem ligar à rede. A principal função do coordenador é gerir e supervisionar todos os nós da rede, garantindo uma comunicação e um funcionamento sem problemas [14].

Um **router** ZigBee é um dispositivo intermediário dentro da rede. Tem a função de encaminhar pacotes de dados entre dispositivos, assegurando que os dados viajam desde a sua origem até ao seu destino. Para além de transmitirem dados, os routers ZigBee também monitorizam o ambiente circundante para detetar a entrada de dados de dispositivos próximos. Isto permite-lhes manter uma ligação de rede estável, fornecendo rotas de transmissão de dados fiáveis [14].

Um **dispositivo final (*end device*)** pode ser um dispositivo de baixa potência ou alimentado por bateria com capacidades de processamento limitadas. Estes dispositivos podem funcionar como FFDs ou RFDs. Os dispositivos finais recolhem dados de várias fontes, como sensores e interruptores, mas não têm a capacidade de transmitir dados a outros dispositivos por si próprios. Em vez disso, dependem dos seus dispositivos-mãe (um coordenador ou um router) para retransmitir os dados. A simplicidade dos dispositivos finais contribui para uma redução dos custos. Ao contrário dos FFD e dos routers, os dispositivos terminais não precisam de estar constantemente ativos, o que ajuda a poupar energia. Normalmente, cada dispositivo final pode ligar-se a um máximo de 240 nós finais na rede [14].

## 6.3 Topologias de Rede

Esta tecnologia pode basear-se em três topologias de rede, sendo elas estrela (star topology), malha (*mesh topology*) ou árvore (*cluster tree topology*). A topologia de estrela consiste na existência de um nó central (coordenador) conectado a vários nós periféricos (dispositivos finais), não havendo conexão direta entre estes, isto é, toda a comunicação passa pelo nó central. Esta configuração apresenta baixa latência na transmissão de dados, mas é limitada a uma área de cobertura de um único salto e tem pouca fiabilidade, já que uma falha num dos nós interrompe toda a comunicação. A topologia malha permite que os nós comuniquem diretamente entre si ou por múltiplos saltos, melhorando a comunicação devido à redundância de rotas, para além de permitir uma maior cobertura. No entanto, o consumo de energia é maior porque os nós intermediários têm a necessidade de se manter ativos para permitir transmissão dos dados. Por fim, a topologia de árvore combina características de ambas as topologias referidas anteriormente. O coordenador atua como raiz e os nós estão organizados hierarquicamente em camadas, facilitando, desta forma, a monitorização. No entanto, as rotas únicas podem falhar e causar congestionamento próximo ao coordenador [15].

## 6.4 Módulo Xbee

O módulo Xbee é projetado para fornecer suporte a redes sem fio robustas com base no padrão ZigBee, utilizando protocolos específicos para operar em redes sem necessidade de uma infraestrutura permanente, permitindo, assim, a criação de redes organizadas hierarquicamente [16]. Cada módulo pode atuar como coordenador, router ou dispositivo final, dependendo da sua configuração de rede. Utilizam dois parâmetros essenciais para a comunicação entre si: o canal (CH) e o identificador de rede (ID). O CH determina a frequência de comunicação dentro da rede, enquanto o ID assegura que os XBee's que compartilham o mesmo valor de ID pertençam à mesma rede, permitindo a troca de dados entre eles. Para que um módulo XBee comunique com outro, ambos devem estar configurados com o mesmo canal e ID. Caso contrário, a

transmissão de dados não será possível. Por outro lado, os parâmetros DH (*Destination High*) e DL (*Destination Low*), Fig. 15 **a)**, são usados para especificar o endereço de destino de um módulo XBee, permitindo que um módulo envie dados especificamente para outro, identificado por esse endereço único. O módulo XBee pode ser conectado a um Arduino (Fig. 15 **b)**) ou diretamente a um computador via USB, sendo necessário, em alguns casos, retirar o microcontrolador [16].

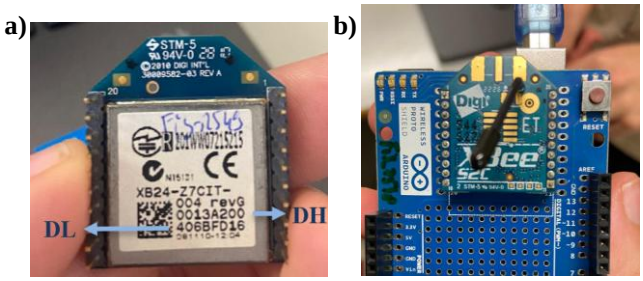



**Fig. 15.** Módulo Xbee **a)** e parâmetros DH e DL, respetivos, e, **b)** no Arduino.

## 6.5 UART

A combinação de UART (*Universal Asynchronous Receiver-Transmitter*) com a tecnologia ZigBee, particularmente através da utilização de módulos XBee, tornou-se uma abordagem padrão para a comunicação sem fios em sistemas incorporados e aplicações IoT. Os módulos XBee, compatíveis com o protocolo IEEE 802.15.4, facilitam a implementação de redes ZigBee, oferecendo conetividade sem fios de baixo consumo, fiável e fácil de configurar. Estes módulos utilizam a UART como interface principal para a troca de dados com microcontroladores, colmatando a lacuna entre a comunicação em série com fios e a transmissão sem fios. A UART também oferece suporte à transmissão transparente, na qual os dados são enviados da linha TX de um dispositivo e recebidos pela linha RX de outro dispositivo sem qualquer modificação ou interpretação. Amplamente utilizado em sistemas de comunicação que exigem a transferência direta de dados [17].

Os comandos AT são utilizados para controlar modems e outros dispositivos através do envio de instruções específicas. São essenciais para a configuração dos parâmetros de comunicação, como a velocidade de transmissão, o tipo de paridade e o formato dos dados nos sistemas UART. No contexto dos módulos XBee, os comandos AT podem ser enviados através de pacotes API para configurar módulos remotos, permitindo um controlo flexível e dinâmico das definições do dispositivo. As API são cruciais no desenvolvimento de *software*, permitindo a integração e a interação de diferentes componentes de *software*. Estas evoluíram significativamente, com milhões de API disponíveis em várias plataformas. Os comandos UART e AT desempenham um papel vital na comunicação por *hardware*, fornecendo interfaces fiáveis e configuráveis para a troca de dados. Inovações recentes melhoraram a sua adaptabilidade e facilidade de utilização. No entanto, as API continuam a ser essenciais no desenvolvimento de *software*, com a investigação em curso a abordar os desafios da evolução e documentação das API [18][19].

A UART permite a transferência contínua de dados em série entre o microcontrolador e o módulo XBee, que depois converte os dados em pacotes ZigBee para comunicação sem fios. Este processo reduz a complexidade da configuração de redes ZigBee, uma vez que a UART requer apenas ligações básicas de TX, RX e GND para funcionar. Tirando partido da interface UART, os programadores podem configurar e monitorizar módulos XBee, enviar comandos e obter dados de sensores ou atuadores sem necessidade de protocolos de comunicação avançados. A abordagem é amplamente adotada em aplicações como a monitorização ambiental, em que os dados devem ser recolhidos de vários nós remotos de forma eficiente [17].

Os módulos XBee numa rede ZigBee podem servir como coordenadores, routers ou dispositivos finais, proporcionando flexibilidade na topologia da rede. Por exemplo, os coordenadores ligam-se frequentemente a um sistema de base através de UART para gerir a rede e transmitir informações críticas. Os dispositivos finais, especialmente os que funcionam com bateria, utilizam a UART para fazer a interface com sensores ou atuadores e dependem dos seus dispositivos-mãe (encaminhadores ou coordenadores) para a transmissão de dados dentro da rede. Esta divisão de funções, juntamente com a natureza direta da comunicação UART, assegura uma utilização eficiente dos recursos e a escalabilidade da rede [17].

A integração de UART e ZigBee tem de demonstrado eficaz, como em aplicações de módulos XBee em sistemas de deteção de incêndios. Os módulos XBee simplificam a comunicação de dados, reduzem a complexidade do *hardware* e suportam um funcionamento com baixo consumo de energia, o que os torna ideais para sistemas IoT que requerem uma conetividade sem fios robusta e escalável [17].

## 6.6 X-CTU

O programa X-CTU é um *software* especializado desenvolvido pela Digi International para configurar e testar rádios XBee em redes ZigBee, funcionando exclusivamente em Windows. Inicialmente, o X-CTU era utilizado principalmente para carregar o *firmware* nos módulos XBee, garantindo a compatibilidade com os protocolos de rede desejados. Atualmente, serve um objetivo mais amplo, oferecendo uma variedade de ferramentas para gerir as tarefas de configuração e diagnóstico associadas aos dispositivos XBee, como a configuração de parâmetros de comunicação, o teste de conetividade e a monitorização da intensidade do sinal.

O X-CTU simplifica o processo de atualizações de *firmware*, garantindo que os rádios XBee funcionam corretamente e podem desempenhar funções como coordenador, router ou dispositivo final numa rede ZigBee. Além disso, o *software* fornece diagnósticos para testar e verificar o desempenho da rede, oferecendo feedback em tempo real sobre a qualidade da comunicação e a saúde do sistema. Isto é particularmente útil para os programadores que trabalham em ambientes complexos onde é necessário configurar e manter vários módulos XBee.

Ao tirar partido das configurações UART no X-CTU, os utilizadores podem afinar os parâmetros de comunicação entre os módulos XBee e o sistema anfitrião, facilitando a transmissão fiável de dados em aplicações integradas. Isto garante que os rádios comunicam eficazmente com microcontroladores ou outros dispositivos, melhorando a funcionalidade geral da rede ZigBee [20].

## 6.7 Configuração rede Zigbee

O primeiro passo da configuração da rede ZigBee passa por adicionar o módulo XBee ao XCTU, um *software* usado para configurar, testar e gerir módulos XBee e dispositivos de comunicação sem fio. Deste modo, liga-se o Xbee ao computador via USB e, no *software*, em “Discover Radio Modules” escolhe-se a respetiva porta USB, pelo que o sistema irá procurar por módulos conectados à mesma. Após encontrar o mesmo, este é adicionado à tela principal, e é aberta uma janela que possui as configurações previamente gravadas no módulo bem como outros parâmetros relevantes para a comunicação. Assim, primeiramente, destacam-se o PAN ID, o canal de operação (CH) e o endereço MAC do módulo. O PAN ID assume-se como um identificador da rede para conectar dispositivos na mesma rede, sendo que é imperativo que este seja igual para todos os módulos nessa rede. O CH, por sua vez, determina a frequência de rádio que é usada para a comunicação, devendo também ser igual em todos os XBee. Já o endereço MAC, por sua vez, é o endereço físico exclusivo do módulo, sendo que o SH (*Serial Number High*) determina os 32 bits mais significativos, enquanto que o SL (*Serial Number Low*) os 32 bits menos significativos. Por fim, é importante realçar que todos os parâmetros alterados devem ser guardados de modo a que a configuração seja eficiente, algo que é feito através da opção “Write”.

### 6.7.1 Configuração do Router

Numa rede ZigBee um router encaminha pacotes de dados entre *end-devices* e o coordenador da rede, estando constantemente em comunicação com este último. Assim, de modo a configurar o módulo como router, o procedimento inicial é análogo ao descrito para o coordenador, apenas com a diferença de selecionar a função “ZigBee Router API”. De seguida, é crucial ajustar alguns parâmetros, como o PAN ID, previamente definido pelo coordenador, e os endereços DH (Destination High) e DL (Destination Low) do coordenador, usados para configurar o endereço de destino de um pacote de dados em redes ZigBee, que especificam os 32 bits mais e menos significativos, respetivamente, do endereço de destino.

Após todas as configurações estarem ajustadas corretamente, é necessário fazer upload do código no Arduino IDE, removendo o Shield para que o Arduino comunique com o computador. Após o código ser carregado, é essencial o interruptor do Shield encontrar-se na posição Serial e, após todas estas etapas estarem concluídas, a comunicação deverá realizar-se sem qualquer tipo de problema.

### 6.7.2 Configuração do Coordenador

O coordenador assume-se como o elemento central na comunicação ZigBee, dado que este é responsável por gerir a rede e todos os outros dispositivos (routers) devem se conectar ao coordenador. A configuração de um módulo como coordenador começa por especificar ao *software* que o XBee em uso vai assumir o papel de coordenador. Deste modo, em “Update Firmware” seleciona-se a família do produto e a função do mesmo como “ZigBee Coordinator API”, escolhendo-se ainda a versão mais recente do firmware. De

seguida, seleciona-se um valor para o PAN ID e regista-se o endereço do router com o qual o coordenador estabelecerá comunicação, modificando-se o DH e o DL. Por fim, seleciona-se a opção “Write” de modo a que as modificações efetuadas sejam guardadas.

### 6.8 Exemplo de rede implementado

De modo a que fosse possível implementar todos os conteúdos expostos nesta secção, foi implementada uma rede na qual 6 routers recolhiam, a partir de localizações próximas mas diferentes, dados de temperatura e humidade, através de um sensor DHT11, e enviavam posteriormente para um coordenador previamente definido.

Deste modo, cada router introduziria o endereço do coordenador no XCTU, pelo que todos teriam de partilhar o mesmo PAN ID. Após a realização desta etapa e o carregamento do respetivo código de modo a enviar os parâmetros da temperatura e da humidade, seria possível visualizar no *software* todos os dispositivos em comunicação ZigBee na rede, evidenciado na Figura 16.

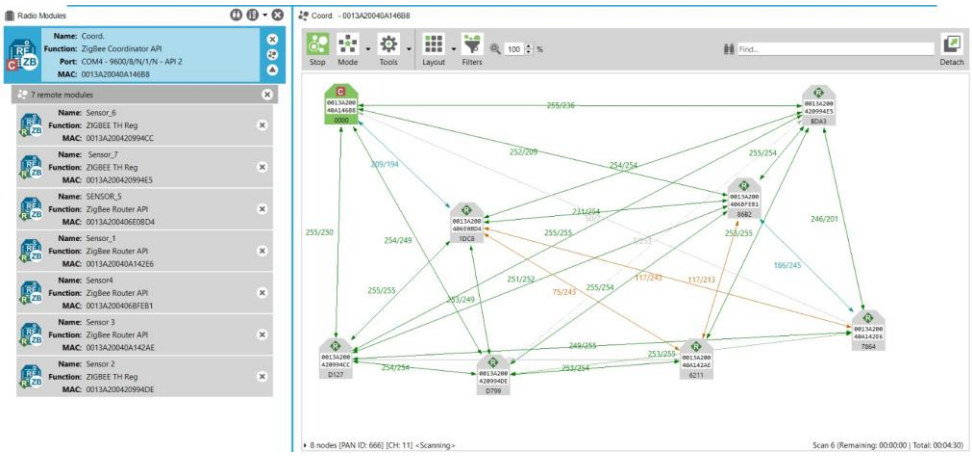



**Fig. 16.** Rede implementada.

## 7 Esp32

O ESP32 (Fig. 17) é um microcontrolador de baixo custo e alta performance (equipado com processador *Ultra-Low-Power*), comummente utilizado em aplicações de IoT, desenvolvido pela *Espressif Systems*. Integra conectividade Wi-Fi e Bluetooth, o que permite a comunicação sem fio com outros dispositivos e plataformas, sendo compatível com *Bluetooth Classic* e *Bluetooth Low Energy*. Tem uma arquitetura de 32 *bits*, o que permite a realização de operações rápidas e complexas, ideal para projetos que exigem automação eficiente e flexibilidade. Tem 48 pinos de entrada e saída, suportando uma grande variedade de sensores e dispositivos [21].

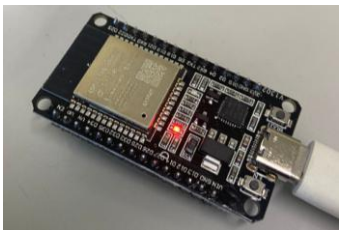


**Fig. 17.** Microcontrolador ESP32.

A sua arquitetura *dual-core* inclui dois microprocessadores *Xtensa LX6* de 32 *bits*, que operam em velocidades de até 240 MHz, garantindo alta uma capacidade de processamento. Este microcontrolador possui uma interface versátil para entrada e saída de dados, suportando periféricos como GPIOs, UART, I2C, SPI, ADCs, DACs e PWM, facilitando a integração com sensores e atuadores, e, é compatível com Arduino. Assim, o Arduino IDE é muito utilizado para programar o ESP32, o que permite explorar recursos avançados do mesmo [21]. Para tal, é necessário a incorporação do suporte para as placas ESP32 no IDE (Fig. 18) tendo-se acedido a "*File*" e, de seguida, "*Preferences*", completando com a informação necessária (Fig. 18).

Posteriormente, em "*Boards Manager*" procedeu-se à instalação do "*ESP32 by Espressif Systems*". Assim, a placa ESP32 ficou disponível para seleção.

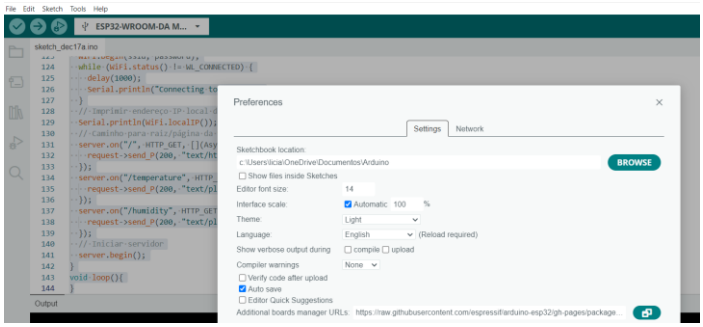



**Fig. 18.** Configuração do ESP32 no Arduino IDE.

### 7.1 Exemplo teste

Depois da ESP32 ficar disponível para seleção no Arduino IDE, realizou-se um teste através da seleção de *Examples* < *WiFi* < *WiFiScan* respetivamente, em "*File*", como podemos ver na Fig. 19. Desta forma, foram apresentadas as redes disponíveis por perto.

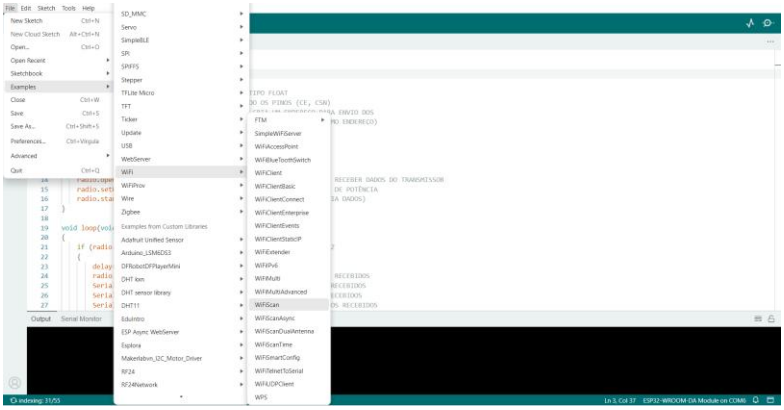



**Fig. 19.** Exemplo teste ESP32 no Arduino IDE.

## 8 Comunicação Wireless Wi-Fi

A comunicação Wi-Fi, nome associado ao protocolo IEEE 802.11, revolucionou por completo a forma como os dispositivos se conectam à Internet, proporcionando uma mobilidade e conveniência que, até então, não existiam, permitindo a transmissão de dados sem a necessidade de cabos físicos. Ao operar em frequências específicas, como 2.4 GHz e, mais recentemente, 5 GHz, o Wi-Fi garante diversas velocidades para atender às diferentes necessidades [22].

### 8.1 Servidor Web ESP32 assíncrono com o DHT11

Um servidor Web ESP32 assíncrono com o sensor DHT11 combina a capacidade do ESP32 de se conectar à rede Wi-Fi com a funcionalidade do sensor DHT11, que mede temperatura e humidade, permitindo criar um sistema de monotorização em tempo real onde os dados obtidos são exibidos numa página web hospedada pelo próprio ESP32. A conexão do sensor DHT11 à placa ESP32 é realizada através do pino de saída de dados do sensor, que está ligado ao pino digital 27 do ESP32. Além disso, o DHT11 deve ser alimentado e o pino GND deve ser conectado à massa (GND). Uma vez realizada a conexão física, é necessário carregar o código fornecido no ESP32 através do Arduino IDE, garantindo que as bibliotecas necessárias estejam instaladas. No código, o nome da rede Wi-Fi e a senha devem ser configurados corretamente para que o ESP32 consiga conectar-se à rede. Quando a conexão é estabelecida, o endereço IP do dispositivo é exibido no serial monitor, permitindo que o servidor seja acedido por um navegador da web.

O servidor web criado utiliza o protocolo HTTP na porta 80 e implementa funções assíncronas para responder rapidamente aos pedidos dos clientes. Os valores de temperatura e humidade são lidos pelo sensor DHT11 e atualizados na página web a cada 10 segundos, proporcionando uma experiência de monotorização em tempo real, tal como evidenciado na Figura 20.

**Fig. 20.** Exemplo do servidor WEB providenciado pelo ESP32 com os valores de temperatura e humidade.

## 8.2 NRF24L01 com o DHT11

O NRF24L01 é um transcetor sem fio de baixa potência, produzido pela Nordic Semiconductor, que opera na faixa dos 2.4 GHz, sendo comumente utilizado em aplicações que requeiram comunicações *wireless* de curto alcance [23]. Este módulo é reconhecido pelo seu alcance de até 50 metros e uma taxa de transmissão adequada a esta aplicação. A taxa de transmissão de dados pode alcançar os 2 Mbps, e comunica via *Serial Peripheral Interface*, facilitando a sua integração com outros microcontroladores como o Arduino [24]. Este dispositivo possui ainda a capacidade de receber sinais de até 6 transmissores sem que existam interferências entre os mesmos [25]. Os códigos representados na Figura 21 a) e b) implementam a comunicação sem fio utilizando o NRF24L01, juntamente com o sensor de temperatura e humidade DHT11. Estes estão organizados em dois módulos: transmissor (TX) e recetor (RX).

**a)**

```
#include <SPI.h>                    //INCLUSÃO DE BIBLIOTECA
#include <nRF24L01.h>               //INCLUSÃO DE BIBLIOTECA
#include <RF24.h>                   //INCLUSÃO DE BIBLIOTECA

float temp[2]={0,0};                //CRIA VETOR DO TIPO FLOAT
RF24 radio(9, 10);                  //CRIA UMA INSTÂNCIA UTILIZANDO OS PINOS (CE, CSN)
const uint64_t pipe= 0xE8E8F0F0E1LL;            //CRIA UM ENDEREÇO PARA ENVIO DOS
//DADOS (O TRANSMISSOR E O RECEPTOR DEVEM SER CONFIGURADOS COM O MESMO ENDEREÇO)

void setup(void)
{
 Serial.begin(9600);                //INICIALIZA A PORTA SERIE DO ARDUINO
 radio.begin();                     //INICIALIZA A COMUNICAÇÃO SEM FIO
 radio.openReadingPipe(0,pipe); /   //DEFINE O ENDEREÇO PARA RECEBER DADOS DO TRANSMISSOR
 radio.setPALevel(RF24_PA_HIGH);    //DEFINE O NÍVEL DO AMPLIFICADOR DE POTÊNCIA
 radio.startListening();            //DEFINE O MÓDULO COMO RECEPTOR (NÃO ENVIA DADOS)
}
void loop(void)
{
 if(radio.available()){             //SE A COMUNICAÇÃO ESTIVER HABILITADA, FAZ
 {
 radio.read(&temp, sizeof(temp));   //LÊ OS DADOS RECEBIDOS
 Serial.print("Temperatura: ");     //IMPRIME NA SERIAL OS DADOS RECEBIDOS
 Serial.print(temp[0]);             //IMPRIME NA SERIAL OS DADOS RECEBIDOS
 Serial.println(" graus Celsius");  //IMPRIME NA SERIAL OS DADOS RECEBIDOS
 Serial.print("Humidade: ");        //IMPRIME NA SERIAL OS DADOS RECEBIDOS
 Serial.print(temp[1]);             //IMPRIME NA SERIAL OS DADOS RECEBIDOS
 Serial.println("%");               //IMPRIME NA SERIAL OS DADOS RECEBIDOS
 }
}
```

**b)**

```
#include <nRF24L01.h>               //INCLUSAO DE BIBLIOTECA
#include <RF24.h>                   //INCLUSÃO DE BIBLIOTECA
#include <DHT11.h>                  //INCLUSÃO DE BIBLIOTECA SENSOR DHT11

int pin = A0;                       // CHAMA PIN AO PINO A0 DO TIPO INTEIRO
DHT11 dht11(pin);                   // INDICA PIN COMO PINO DE DADOS DO SENSOR DHT11
float temperature[2];               //CRIA VETOR DO TIPO FLOAT

RF24 radio(9, 10);                  //CRIA UMA INSTÂNCIA UTILIZANDO OS PINOS (CE, CSN)
const uint64_t pipe = 0xE8E8F0F0E1LL;  //CRIA UM ENDEREÇO PARA ENVIO DOS
//DADOS (O TRANSMISSOR E O RECEPTOR DEVEM SER CONFIGURADOS COM O MESMO ENDEREÇO)

 void setup(void) {
Serial.begin(9600);                 //INICIALIZA A PORTA SERIE DO ARDUINO
radio.begin();                      //INICIALIZA A COMUNICAÇÃO SEM FIO
radio.openWritingPipe(pipe);        //DEFINE O ENDEREÇO PARA ENVIO DE DADOS AO RECEPTOR
radio.setPALevel(RF24_PA_HIGH);     //DEFINE O NÍVEL DO AMPLIFICADOR DE POTÊNCIA
radio.stopListening();              //DEFINE O MÓDULO COMO TRANSMISSOR (NÃO RECEBE DADOS)
}
 void loop(void) {
    float temp, humi;               //INICIA FLOAT
    dht11.read(humi, temp);         //LÊ DO SENSOR MSG A SER TRANSMITIDA
    temperature[0] = temp;          //ATRIBUI VALORES A LINHA DO VETOR DE FLOAT
    temperature[1] = humi;          //ATRIBUI VALORES A LINHA DO VETOR DE FLOAT
    Serial.print("Temperatura:");   //ESCREVE NA PORTA SERIE DO ARDUINO
    Serial.print(temperature[0]);   //ESCREVE NA PORTA SERIE DO ARDUINO
    Serial.println(" °C ");         //ESCREVE NA PORTA SERIE DO ARDUINO
    Serial.print("Humidade: ");     //ESCREVE NA PORTA SERIE DO ARDUINO
    Serial.print(temperature[1]);   //ESCREVE NA PORTA SERIE DO ARDUINO
    Serial.println(" %");           //ESCREVE NA PORTA SERIE DO ARDUINO
    radio.write(&temperature, sizeof(temperature));        //ENVIA AO RECEPTOR A MENSAGEM
    delay(2000);                    //INTERVALO DE 2 SEGUNDOS
}
```

**Fig. 21.** Códigos para o RX a) e TX b).

No módulo TX, o Arduino está conectado ao sensor DHT11 para recolher os dados de temperatura e humidade. Após a leitura, os valores são enviados via NRF24L01 ao RX. O código inclui as bibliotecas nRF24L01 (para comunicação sem fio) e DHT.h (para o sensor DHT11). A cada 2 segundos, os dados recebidos são transmitidos. Já no módulo RX, o Arduino recebe os dados enviados pelo transmissor usando o NRF24L01. O recetor monitoriza continuamente se existem dados disponíveis e, ao receber as informações, processa e exibe os valores de temperatura e humidade no *serial monitor*. Assim, o módulo recetor utiliza apenas a biblioteca nRF24L01, pois sua função é exclusivamente a receção e exibição dos dados.

## 8.3 Principais Diferenças entre Wi-Fi do ESP32 vs NRF24L01

O módulo NRF24L01 foi implementado em conjunto com o sensor DHT11 para monitorizar temperatura e humidade, utilizando um esquema de comunicação transmissor-recetor. No transmissor, o NRF24L01 foi conectado a um Arduino, que processava os dados provenientes do sensor e os enviava periodicamente. O recetor foi, por sua vez, configurado para receber e exibir os dados no monitor serial. Os testes realizados, na Figura 22, demonstraram o funcionamento eficiente do sistema, com transmissão estável e sem perdas significativas, comprovando a eficácia do NRF24L01 em comunicações sem fio de curto alcance e baixo consumo energético. Contudo, a ausência de suporte a redes *Transmission Control Protocol/Internet Protocol* (TCP/IP) limita a sua aplicação em projetos que exigem conectividade à internet.

Apesar das limitações do NRF24L01, o ESP32 destaca-se como uma solução robusta para aplicações que necessitam de conectividade avançada, como sistemas de automação residencial ou monitorização remota no contexto da IoT. Integrando conectividade Wi-Fi nativa e suporte a redes TCP/IP, o ESP32 possui maior capacidade de processamento, memória e compatibilidade com múltiplos protocolos de comunicação. Contudo, a simplicidade e eficiência energética do NRF24L01 mantêm-no relevante em projetos que

não demandam conexão à internet, mas sim a redes locais, como sistemas ponto a ponto ou de malha. Na sua configuração amplificada, o NRF24L01 consegue alcançar distâncias de até 1 km em linha de visão, com taxas de transmissão de 2 Mbps, enquanto o ESP32 opera com taxas de até 150 Mbps, atingindo cerca de 100 metros em áreas abertas [26].

A integração entre o NRF24L01 e o ESP32 revela-se particularmente interessante em cenários específicos como a comunicação veicular e monitorização IoT. No contexto da comunicação veículo-a-veículo, ambos os dispositivos são empregados em redes *ad-hoc* veiculares, operando em modos simplex ou half-duplex para viabilizar a transmissão de dados em sistemas de mobilidade inteligente. O uso combinado desses componentes permite uma solução de comunicação confiável e de baixo custo, crucial para melhorar a conectividade e a segurança em veículos. De forma semelhante, na monitorização IoT, como a análise de saúde de rebanhos, o NRF24L01+PA+LNA é utilizado para transmissões de longa distância, enquanto o ESP32 centraliza a recolha e o processamento dos dados provenientes de múltiplos sensores, disponibilizando informações em tempo real para visualização em dispositivos móveis[26].

Do ponto de vista de desempenho, o consumo energético é uma diferença marcante entre os dois componentes. O NRF24L01, com apenas 22 µA em estado de espera e 11,3 mA durante transmissão, é especialmente vantajoso em dispositivos alimentados por bateria. Em contrapartida, o ESP32, apesar de oferecer maior versatilidade, apresenta maior consumo, variando entre 10 µA em modo de repouso profundo e até 0,8 mA em operação reduzida. No entanto, o ESP32 compensa com técnicas avançadas como computação distribuída, sensoriamento hierárquico e *duty cycling*, que ajudam a otimizar a eficiência energética em redes IoT onde a autonomia das baterias é essencial [27].

Embora o ESP32 se destaque em termos de conectividade e capacidade de processamento, o NRF24L01 continua a ser uma opção atrativa devido ao seu custo reduzido e facilidade de integração. A escolha entre os dois dispositivos depende das necessidades específicas de cada projeto, como o equilíbrio entre custo, alcance, consumo energético e exigências de conectividade, sendo comum que os dois dispositivos sejam utilizados de forma complementar para maximizar a eficiência e a funcionalidade em sistemas de comunicação sem fio.

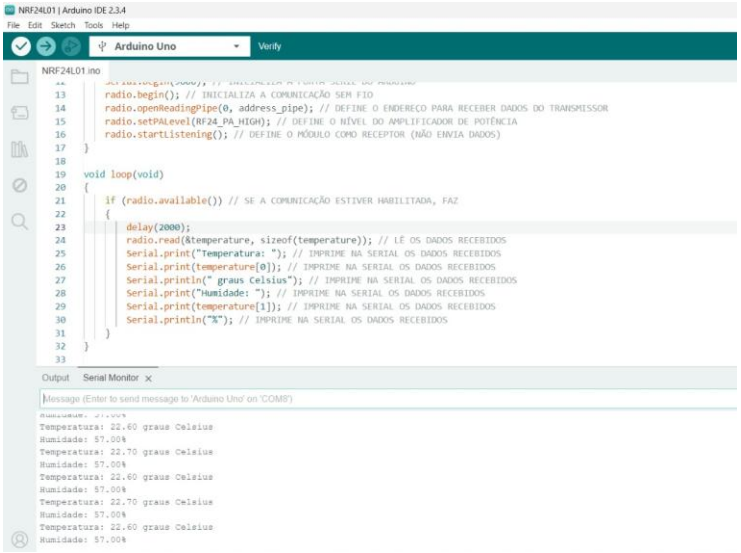



**Fig. 22.** Exemplo de teste NRF24L01 no Arduino IDE.

## 9 Comunicação Wireless Bluetooth

A comunicação Wireless Bluetooth é uma tecnologia sem fio com um alcance muito curto, projetada para permitir a comunicação entre dispositivos como computadores e outros dispositivos sem recorrer a cabelagem ou conectores [28]. Além disso, o facto de consumir um nível residual de energia faz com que seja ideal para sensores alimentados por baterias. O *Bluetooth Low Energy* (BLE) é uma versão aprimorada e mais eficiente do Bluetooth clássico, projetada para aplicações com um baixo custo de energia, como dispositivos como sensores de batimento cardíaco, pedómetros, sensores para bicicleta entre outros [29].

## 9.1 ESP32 + DHT11 (Bluetooth Low Energy)

O ESP32 permite ler dados de temperatura e humidade de um sensor DHT11 e transmiti-los via BLE para uma aplicação. O processo envolve conectar o sensor ao ESP32 e programar o microcontrolador para atuar como um servidor BLE que disponibiliza os dados a uma aplicação, que pode ser concebida em várias plataformas como o Thunkable ou o MIT APP INVENTOR.

A configuração inicial envolve a presença dos *Universally Unique Identifiers* (UUIDs), que são identificadores únicos utilizados para distinguir serviços e características específicas dentro da comunicação BLE, permitindo que dispositivos reconheçam e acedam corretamente aos dados transmitidos pelo ESP32, como temperatura e humidade recolhidos pelo sensor DHT11.

## 9.2 Integração do Módulo HC-05 + Arduino + DHT11 e App Android

Primeiramente, é necessário ligar o módulo HC-05 ao Arduino, especificamente aos pinos RX e TX do mesmo, e o DHT11 à alimentação / massa e a um pino previamente definido no Arduino.

Seguidamente, o Arduino consegue ler os dados enviados pelo sensor DHT11 e, através da comunicação Serial, reencaminhá-los para o módulo HC-05 que, por sua vez, estabelece uma comunicação sem fio com a aplicação Android, permitindo ver os valores da humidade e temperatura em tempo real. De modo a essa comunicação ser estabelecida, é necessário que o dispositivo Android emparelhe, via Bluetooth, com o módulo HC-05, introduzindo o devido PIN de emparelhamento. O resultado final está visível na Figura 23.

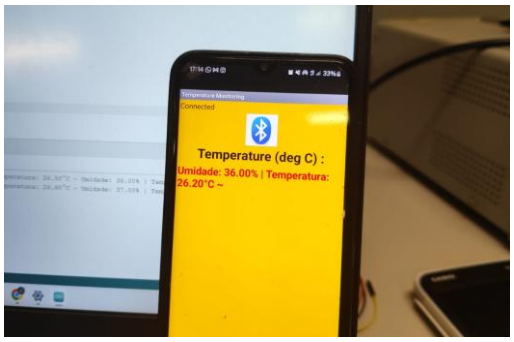



**Fig. 23.** Valores de humidade e temperatura exibidos na App Android.

## 9.3 Principais Diferenças entre Bluetooth de ESP32 vs HC-05

O ESP32 e o HC-05 são dispositivos amplamente utilizados para comunicação Bluetooth, mas atendem a necessidades diferentes pelo que apresentam diferenças em termos de funcionalidades e aplicações.

O HC-05 é um módulo Bluetooth simples e tradicional (mais barato que o HC-05) de fácil utilização, projetado para configurações de conexão serial sem fio por meio do protocolo SPP (*Serial Port Protocol*). Este módulo suporta Bluetooth V2.0+EDR (*Enhanced Data Rate*) com uma modulação de 3 Mbps na faixa de 2.4 GHz, podendo ser configurado como mestre ou escravo, sendo, deste modo, adequado para comunicações ponto a ponto entre dispositivos simples. Contudo, o HC-05 não suporta *Bluetooth Low Energy*, o que limita a sua eficiência em aplicações que exigem baixo consumo de energia [30]. Por outro lado, o ESP32 oferece suporte tanto para Bluetooth clássico (*Basic Rate/Enhanced Data Rate*) como para BLE. Deste modo, esta abrangência permite uma compatibilidade mais ampla com dispositivos e aplicações, especialmente em cenários de IoT. Com Bluetooth 4.2, o ESP32 pode alcançar velocidades de transmissão de até 4 Mbps e é mais eficiente em dispositivos modernos que priorizam baixo consumo de energia, tornando-o uma escolha superior para aplicações mais avançadas, comparado ao HC-05, que se restringe a comunicações ponto a ponto e não é ideal para cenários que necessitam de eficiência energética [31]. Além disso, pelo facto de ser um microcontrolador completo, este consegue processar dados sem a necessidade de um externo como o Arduino, algo que não se verifica com o módulo HC-05.

Em suma, o HC-05 é uma solução simples e acessível para comunicação Bluetooth ponto a ponto, mas que não suporta BLE e com limitações em termos de eficiência energética. Já o ESP32 oferece suporte a Bluetooth clássico e BLE, para além de uma maior velocidade de transmissão e eficiência em dispositivos IoT, além de ser ainda um microcontrolador completo, dispensando controladores externos como o Arduíno. Deste modo, o ESP32 é mais indicado para aplicações avançadas e sistemas modernos, enquanto que o HC-05 satisfaz os requisitos de aplicações simples e com baixo custo.

# 10 Conclusão

A realização deste projeto e respetivo relatório permitiu a compreensão e aplicação dos princípios fundamentais da instrumentação biomédica, dado que o objetivo principal de desenvolver um protótipo de um sistema de aquisição de dados (DAQ) para a medição da temperatura, utilizando um sensor NTC, foi atingido com sucesso. A implementação de uma ponte de *Wheatstone* para converter as variações de resistência em tensão, aliada ao uso de amplificadores operacionais como *buffers* e amplificadores diferenciais, assegurou a integridade dos sinais e a adaptação ao intervalo compatível com o conversor analógico-digital (ADC). Esses componentes maximizaram a resolução do sistema, sendo que a calibração, realizada com um termómetro de referência, possibilitou a obtenção de uma curva precisa para a relação tensão-temperatura. A calibração do sistema, uma etapa crucial para a identificação e correção dos erros de medição, garantiu que o equipamento pudesse ser utilizado com precisão nas medidas pretendidas. Os resultados obtidos demonstraram a qualidade do sistema, com um coeficiente de correlação de 0,9925, evidenciando a importância da calibração e do condicionamento de sinal. Contudo, era esperado que aos 50 ºC, o valor da tensão fosse igual a aproximadamente + 5 V, o que não aconteceu. Esta ocorrência pode estar relacionada com eventuais problemas na fonte, ou então no facto de a alimentação aos circuitos integrados ser de precisamente ± 5 V, que corresponde à tensão que se pretendia obter à saída do Sistema de Aquisição de Dados. Torna-se importante realçar que os amplificadores operacionais nunca alcançam a tensão máxima de alimentação na sua saída, pelo que, de modo a solucionar este problema, a utilização de uma tensão de alimentação maior, na ordem, por exemplo, dos ± 15 V (± 10 V já seria o suficiente ), poderia ser uma possível solução para o mesmo, dado que, deste modo, proporcionar-se-ia uma margem de operação que evitasse a saturação e permitir-se-ia que os amplificadores tivessem um sinal de saída próximo da tensão esperada.

Na segunda etapa do projeto, foi integrado o Arduíno, de forma a automatizar os cálculos e assegurar uma maior estabilidade nas medições de humidade e temperatura. Foi ainda feita a implementação de diferentes formas de comunicação sem fio, como ZigBee, Wi-Fi e Bluetooth permitiu comparar resultados entre si. A comunicação ZigBee demonstrou utilidade em redes de sensores distribuídas, enquanto que o Wi-Fi, utilizando o ESP32, permitiu criar um servidor *web* para monitorização em tempo real. O uso do Bluetooth, tanto clássico (HC-05) quanto BLE, destacou-se pela flexibilidade na transmissão de dados para dispositivos móveis.

Concluindo, foi possível atingir o objetivo inicial, bem como a aquisição de experiência prática e uma compreensão aprofundada dos conceitos teóricos e práticos envolvidos na aquisição de dados e na instrumentação biomédica.

# 11 Agradecimentos

Agradecemos aos docentes Joaquim Alves e Nuno Gueiral, cuja orientação e apoio foram fundamentais e imprescindíveis ao longo do desenvolvimento deste projeto, bem como, a disponibilidade dispensada e paciência ao longo do desenvolvimento do mesmo.

# 12 Referências

[1] K. Arshak, G. Lyons, C. Cunniffe, J. Harris, and S. Clifford, "A review of digital data acquisition hardware and software for a portable electronic nose," *Sensor Review*, vol. 23, no. 4, pp. 332–344, 2003, doi: 10.1108/02602280310496854.

[2] J. I. Venceslau de Souto, Á. Barbosa da Rocha, R. N. C. Duarte, and E. de Moura Fernandes, "Design and Implementation of an Embedded Data Acquisition System for Vehicle Vertical Dynamics Analysis," *Sensors 2023, Vol. 23, Page 9491*, vol. 23, no. 23, p. 9491, Nov. 2023, doi: 10.3390/S23239491.

[3] "What Is A Thermistor And How Does It Work?" Accessed: Dec. 26, 2024. [Online]. Available: https://www.omega.com/en-us/resources/thermistor

[4] K. Esenowo Jack, E. O. Nwangwu, I. Agwu Etu, and E. U. Osuagwu, "A Simple Thermistor Design for Industrial Temperature Measurement," *IOSR Journal of Electrical and Electronics Engineering*, vol. 11, no. 05, pp. 57–66, May 2016, doi: 10.9790/1676-1105035766.

[5] "Metrologia – IPQ." Accessed: Dec. 26, 2024. [Online]. Available: https://www.ipq.pt/metrologia/

[6] "ISO 10012:2003 - Measurement management systems — Requirements for measurement processes and measuring equipment." Accessed: Dec. 26, 2024. [Online]. Available: https://www.iso.org/standard/26033.html

[7] IPQ, "Calibração de Padrões de Instrumentos de Medição – IPQ." Accessed: Dec. 26, 2024. [Online]. Available: https://www.ipq.pt/servicos%20ipq/calibracao_instrumentos_medicao/

[8] H. K. Kondaveeti, N. K. Kumaravelu, S. D. Vanambathina, S. E. Mathe, and S. Vappangi, "A systematic literature review on prototyping with Arduino: Applications, challenges, advantages, and limitations," *Comput Sci Rev*, vol. 40, p. 100364, May 2021, doi: 10.1016/J.COSREV.2021.100364.

[9] A. A. Galadima, "Arduino as a learning tool," *Proceedings of the 11th International Conference on Electronics, Computer and Computation, ICECCO 2014*, Dec. 2014, doi: 10.1109/ICECCO.2014.6997577.

[10] В. Морозова, "ARDUINO INTEGRATED DEVELOPMENT ENVIRONMENT," 2018, Accessed: Dec. 26, 2024. [Online]. Available: http://foss.ekit.org.ua

[11] G. S. Ritter, E. O. Cavalheiro, R. B. e Silva, L. D. R. Schmidt, and S. Maldaner, "Medidas de temperatura em ambiente interno usando a Plataforma Arduino," *Ciência e Natura*, vol. 42, p. e35, Feb. 2020, doi: 10.5902/2179460X40637.

[12] M. Zimamul Adli, E. Suratman, and M. A. Firdaus, "Comparison of DHT 11 and NTC Temperature Sensors Based on Internet of Things and Lora Ebyte 32 For Incubator," *International Journal of Research and Review*, vol. 11, no. 8, pp. 462–469, Sep. 2024, doi: 10.52403/IJRR.20240849.

[13] M. Ren, X. Ren, H. Feng, J. Ming, and Y. Lei, "Security Analysis of Zigbee Protocol Implementation via Device-agnostic Fuzzing," *Digital Threats: Research and Practice*, vol. 4, no. 1, Mar. 2023, doi: 10.1145/3551894.

[14] H. Jha and S. Kaur, "A Survey on ZigBee Technology," *2023 14th International Conference on Computing Communication and Networking Technologies, ICCCNT 2023*, 2023, doi: 10.1109/ICCCNT56998.2023.10306560.

[15] Y. Yu, B. Xue, Z. Chen, and Z. Qian, "Cluster tree topology construction method based on PSO algorithm to prolong the lifetime of ZigBee wireless sensor networks," *EURASIP J Wirel Commun Netw*, vol. 2019, no. 1, Dec. 2019, doi: 10.1186/s13638-019-1523-0.

[16] H. Saha, S. Mandal, S. Mitra, S. Banerjee, and U. Saha, "Comparative Performance Analysis between nRF24L01+ and XBEE ZB Module Based Wireless Ad-hoc Networks," *International Journal of Computer Network and Information Security*, vol. 9, no. 7, pp. 36–44, Jul. 2017, doi: 10.5815/ijcnis.2017.07.05.

[17] A. Mukherji and S. Sadu, "ZigBee performance analysis," *Proceedings of the 2016 IEEE International Conference on Wireless Communications, Signal Processing and Networking, WiSPNET 2016*, pp. 325–329, Sep. 2016, doi: 10.1109/WISPNET.2016.7566148.

[18] S. Harutyunyan, T. Kaplanyan, A. Kirakosyan, and A. Momjyan, "Design and Verification of Autoconfigurable UART Controller," *Proceedings - IEEE International Conference on Electronics and Nanotechnology, ELNANO*, pp. 347–350, 2020, doi: 10.1109/ELNANO50318.2020.9088789.

[19] M. Lamothe, Y. G. Guéhéneuc, and W. Shang, "A Systematic Review of API Evolution Literature," *ACM Comput Surv*, vol. 54, no. 8, Nov. 2022, doi: 10.1145/3470133.

[20] H. Kumbhar, "Wireless sensor network using xbee on arduino platform an experimental study," *Proceedings - 2nd International Conference on Computing, Communication, Control and Automation, ICCUBEA 2016*, Feb. 2017, doi: 10.1109/ICCUBEA.2016.7860081.

[21] G. P. Pereira, M. Z. Chaari, and F. Daroge, "IoT-Enabled Smart Drip Irrigation System Using ESP32," *Internet of Things*, vol. 4, no. 3, pp. 221–243, Sep. 2023, doi: 10.3390/iot4030012.

[22] *2009 4th IEEE Conference on Industrial Electronics and Applications : 25-27 May 2009, Xi'an, China*. IEEE, 2009.

[23] *Proceedings of the IEEE International Conference on Automation and Logistics : Qingdao, China September 2008*. IEEE Xplore, 2008.

[24] *2011 International Conference on Business Management and Electronic Information*. IEEE, 2011.

[25] N. Gueiral, "Modulo NRF24L01 (RF)," Porto, 2024.

[26] A. Ren, H. Shi, and H. Wang, "Multi-point wireless temperature monitoring system based on NRF24L01," p. 109, Sep. 2022, doi: 10.1117/12.2646917.

[27] V. Nkemeni, F. Mieyeville, and P. Tsafack, "Energy Consumption Reduction in Wireless Sensor Network-Based Water Pipeline Monitoring Systems via Energy Conservation Techniques," *Future Internet*, vol. 15, no. 12, Dec. 2023, doi: 10.3390/FI15120402.

[28] M. Verma, S. Singh, and B. Kaur, "An Overview of Bluetooth Technology and its Communication Applications International Journal of Current Engineering and Technology An Overview of Bluetooth Technology and its Communication Applications," 2015. [Online]. Available: http://inpressco.com/category/ijcetGeneralArticle

[29] N. Gueiral, "Exemplo ESP32 + DHT11 (BLUETOOTH LE)," Porto, 2024.

[30] D. Devikanniga, A. Ramu, and A. Haldorai, "Efficient diagnosis of liver disease using support vector machine optimized with crows search algorithm," *EAI Endorsed Transactions on Energy Web*, vol. 7, no. 29, 2020, doi: 10.4108/EAI.13-7-2018.164177.

[31] *2017 Internet Technologies and Applications (ITA)*. IEEE, 2017.